FP 9 (Haskell 5): Recursion and lazy evaluation

Cael Marquard
mrqcae001@myuct.ac.za
(slides adapted from Dr Sabre Didi)

Recap quiz

1. (1 point) What is the :kind of Int?
   - **a.** `Int`
   - **b.** `*`
   - **c.** `* -> *`
   - **d.** `Type { * }`
2. (2 points) A typeclass is…
   - **a.** Encapsulation of data and behaviour
   - **b.** An anonymous set of functions that can be imported
   - **c.** Encapsulation of supported behaviour only for a type
   - **d.** The `data` type equivalent of an OOP class in Haskell

Aim of this lecture

By the end of this lecture, you should:

- Understand how to use recursion in Haskell
- Understand the difference between lazy and strict evaluation
- Explain the difference between `foldl'` and `foldl`
- Be able to write a version of the Fibonacci function that can (relatively) efficiently compute `fib 1_000_000`
- Know what a “thunk” is

Recursion



- Haskell has an equivalent to a **for loop: `foldl/foldr/foldl'`**
- Haskell **does not have while loops** (the imperative implementation of the so-called “unbounded search operator” **μ**: https://en.wikipedia.org/wiki/Mu_operator)
- What it does have is **recursion**, which is **equivalent** in terms of what it can compute
- Just like **any for loop can become a fold, any while-loop can become a recursive function.**

Recursion as while loops: be not afraid

Recursive functions don't have to be complicated! *Sometimes* it's better to think of them as **while loops where state is passed forward in arguments.**

For instance, take this Python code, which searches for the *next* prime number **after** a given integer, and let's think about how to implement it recursively in Haskell:

```
def next_prime(start):
    x = start + 1
    while not isPrime(x):
        x += 1
    return x
```

Recursion as while loops: be not afraid (2)

We can make our Haskell version of next_prime simply track the existing state (x) through its arguments:

```
def kth_next_prime(start, k):
    x = start
    while k > 0:
        x += 1
        if isPrime(x):
            k -= 1
    return x
```

```
nextPrime :: Int -> Int
nextPrime start = searchPrime $ start + 1
  where
    searchPrime start
      | isPrime start = start
      | otherwise     = searchPrime (start + 1)
```

Let's stop and think: why did we define a subfunction searchPrime?

Recursion as while loops: keeping more state

Now let's try to translate this code which finds the **k**th next prime after the start.
`kth_next_prime(2, 4)` would look past 3, 5, and 7, finally returning 11 (2 isn't included as it has to be k **after** start, not incl start)
`kth_next_prime(4, 4)` would look past 5, 7, 11, returning 13.

```
def kth_next_prime(start, k):
    x = start
    while k > 0:
        x += 1
        if isPrime(x):
            k -= 1
    return x
```

```
kthNextPrime :: Int -> Int -> Int
kthNextPrime start k = searchPrime (start + 1) (k - 1)
  where
    searchPrime start k
      | isPrime start && k == 0 = start
      | isPrime start = searchPrime (start + 1) (k - 1)
      | otherwise = searchPrime (start + 1) k
```

Recursion as for loops

We can also use recursion to neatly express for loops (and, transitively, list combinators and comprehensions).

This is how we would implement `sum` recursively:

```
sum' :: Num a => [a] -> a
sum' [] = 0
sum' (x:xs) = x + sum' xs
```

Let's stop and think about how to implement `map' :: (a -> b) -> [a] -> [b]` recursively.

Introducing the class dog

This is the class dog.

Her name is Rosa and she is a Labrador Retriever who is **ALWAYS** happy… just like you should be while you learn Haskell!

She is going to learn Haskell alongside us and remind us never to give up.



Naturally recursive functions

Some functions are more natural to express recursively. Let's do a simple example of a recursive function together: the Fibonacci function.

fib(0) = 0
fib(1) = 1
fib(n) = fib(n - 1) + fib(n - 2)

```
fib :: Int -> Int
fib 0 = 0
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)

fib 10 -- 55
```

Structure of a recursive function

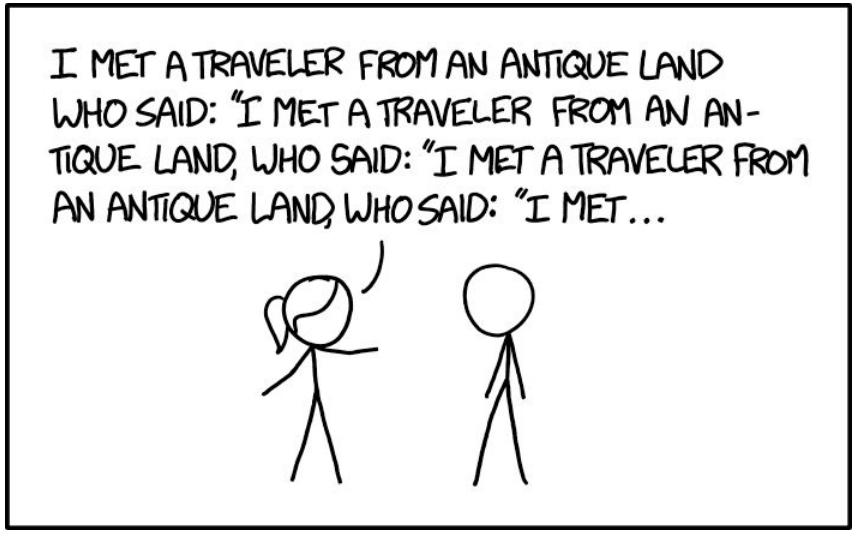
A recursive function must have the following elements (in order to terminate):

- **Recurrence relation**: the bit of the function that calls itself

```
fib n = fib (n - 1) + fib (n - 2)
```

- **Base case:** the bit of the function that defines the output for the *simplest input*, and *stops* the recursion. If you *want* an infinite loop, it's OK to omit this.

```
fib 0 = 0
fib 1 = 1
```



Pictured: someone who forgot her base case!
Randall Munroe, https://xkcd.com/1557/

Naturally recursive functions: quicksort

Many sorting algorithms, such as quicksort, are naturally expressed recursively too.

Quicksort is quite simple.

- [] is already sorted
- Nonempty lists are sorted by picking a pivot and then partitioning:
  - tail = elements <= a pivot
  - head = elements >= a pivot
  - We quicksort our head and tail
  - We then return head + pivot + tail and this is our sorted list

Naturally recursive functions: quicksort (2)

- [] is already sorted
- Nonempty lists are sorted by picking a pivot and then partitioning:
  - tail = elements <= a pivot
  - head = elements > a pivot
  - We quicksort our head and tail
  - We then return head + pivot + tail and this is our sorted list

To simplify things, we'll just pick pivot as the first element of our list every time.

```
qsort :: Ord a => [a] -> [a]
qsort []     = []
qsort (x:xs) = qsort tail ++ [pivot] ++ qsort head
  where
    pivot = x
    tail = [i | i <- xs, i <= pivot]
    head = [i | i <- xs, i >  pivot]

qsort [2, 5, -10, 2]
-- [-10, 2, 2, 5]
```

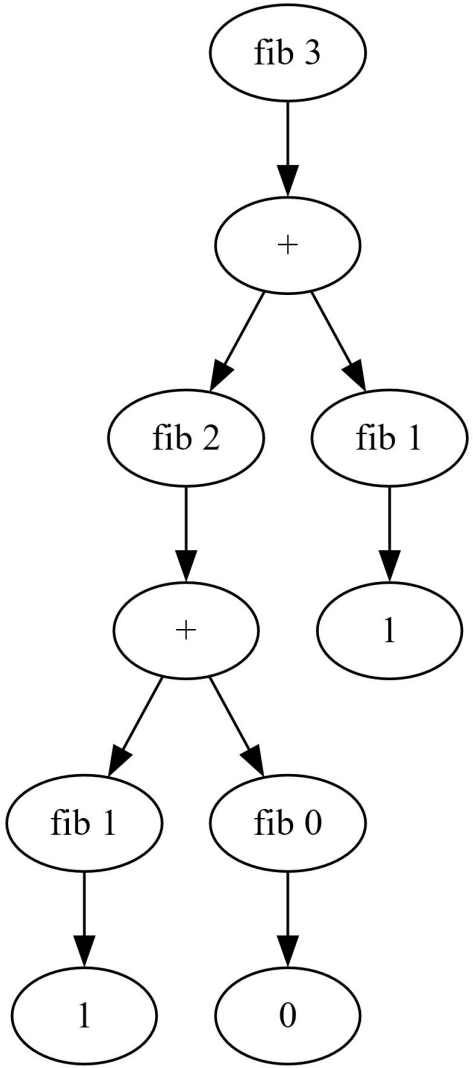
Recursive expansion

```haskell
fib :: Int -> Int
fib 0 = 0
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)

fib 3 -- 3
```

When we call `fib 3`, how does this actually get evaluated? We can find this out by **substituting the function calls for their definitions.**

```
fib 3 = ( fib (3 - 1)            +  fib (3-2)  )
      = ( fib (3 - 1)            +  fib (1)    )
      = ( fib (2)                +      1      )
      = ( fib (2)                +      1      )
      = ( (fib(2-1) + fib(2-2)) +      1      )
      = ( (fib(1) + fib(0))      +      1      )
      = ( (1      +   0   )      +      1      )
      = ( 1  +  1 ) = 2
```



Sorry, everyone, I lied: `foldl` vs `foldl'`

Okay, I lied about `foldl` being a good function to use.

Don't get me wrong, folds are great! But there is a **trap waiting** with **`foldl`** *specifically*: stack overflows.

Let's look at an example:

```
ns = [1..300_000_000]
foldl (+) 0 ns
```



Running this promptly allocates 14.3GiB of memory and may even reward us with
`*** Exception: stack overflow` for our efforts. At a paltry 300 million items? Rude!

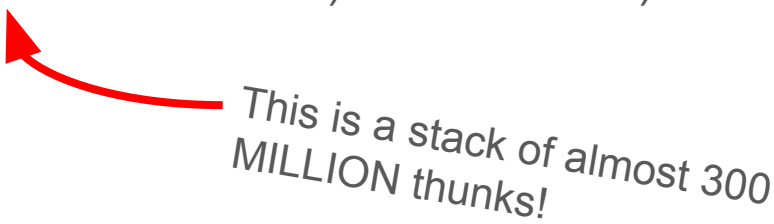
Stack overflows with `foldl`

Why does this happen? Let's return to the expansion.

`foldl (+) 0 ns` is essentially computing ((((0 + 1) + 2) + 3) + 4) + …

Since Haskell follows lazy, **outermost** evaluation, we end up with *this entire thing* (we call unevaluated expressions *thunks* in Haskell) until we get to the final calculation:

((… + 299999998) + 299999999) + 300000000



We cannot reduce our expression until we already run out memory!

The lazy kitchen analogy

… with only one bench!

(see lecture video for explanation)



Rewriting it with `foldr`

We could try rewrite this with foldr and see if this changes (remember: this would NOT be possible for a non-associative/commutative operator like -)

`foldr (+) 0 ns` is essentially computing 1 + (2 + (3 + (4 + …)))

… but this doesn't help us either! Before we resolve 1 + (2 + …), we have to resolve 2 + (3 + …), and before we resolve that… you can see where this is going!

`foldl'` is `foldl` but *strictly applies the operator*

Ultimately, what we need to do is **stop letting Haskell pile up these thunks!**

The way to do this is to **enforce strict evaluation** of our + operator. That way, it collapses *as we go* rather than once, at the end (when the stack is already too big)

This is done with the special (**magic - not user-implementable**) `seq` function:

```
-- Takes two arguments:
-- a: the value to strictly evaluate
-- b: the value to return
-- Strictly evals a, then returns b.
-- It's useful to have b defined in
terms of a
seq :: a -> b -> b
-- example use:
seq arg (function arg)
```

```
-- Lazy
foldl  f z []     = z
foldl  f z (x:xs) = let acc' = f acc x
                    in foldl f acc' xs

-- Strict in operator, f
foldl' f acc []     = acc
foldl' f acc (x:xs) = let acc' = f acc x
                      in seq acc' $ foldl' f acc' xs
```

`foldl'` works!

```
ghci> ns = [1..300_000_000]
ghci> foldr (+) 0 ns
*** Exception: stack overflow
ghci> foldl' (+) 0 ns
45000000150000000
ghci>
```



Think of the thunks!

The most important thing about recursion in Haskell is to **think of the thunks!**

Because Haskell is **lazy***, it doesn't matter if a notional "call stack" becomes too big, since Haskell doesn't really have this.

What matter is keeping the number of unevaluated thunks down, as with our `fold` and `foldl'` example, by judiciously forcing strict evaluation when necessary.

*technically 🤓☝ Haskell is only specified as *non-strict,* which is ever-so-slightly different. See the wiki for more [AR = "Lazy vs Non-strict" on Haskell Wiki]

Think of the thunks: Fibonacci

Let's return to our Fibonacci function. Recall how `fib 3` expands:

```
fib 3 = ( fib (3 - 1)            +  fib (3-2)  )
      = ( fib (3 - 1)            +  fib (1)    )
      = ( fib (2)                +  1          )
      = ( fib (2)                +      1      )
      = ( (fib(2-1) + fib(2-2)) +      1      )
      = ( (fib(1) + fib(0))      +      1      )
      = ( (1      +   0   )      +      1      )
      = ( 1  +  1 ) = 3
```

```
fib :: Int -> Int
fib 0 = 1
fib 1 = 0
fib n = fib (n - 1) + fib (n - 2)

fib 3 -- 2
```

Spot any issues with this?

Think of the thunks: Fibonacci (2)

We are evaluating `fib 1` twice. I hope it is obvious that this problem only gets worse for larger values of `n`…

```
fib 3 = ( fib (3 - 1)            +  fib (3-2)  )

      = ( fib (3 - 1)            +  fib (1)    )

      = ( fib (2)                +  1          )

      = ( fib (2)                +      1      )

      = ( (fib(2-1) + fib(2-2)) +       1      )

      = ( (fib(1) + fib(0))      +       1      )

      = ( (1      +   0   )      +       1      )

      = ( 1  +  1 ) = 3
```

```haskell
fib :: Int -> Int
fib 0 = 1
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)

fib 3 -- 3
```

Memoising Fibonacci

We can fix this with **memoisation:** caching the previously-computed values of a function (this is conceptually similar to what you do in **dynamic programming**).

There are many ways to achieve this. First, I will present one which is familiar to you from Dynamic Programming. Let's follow along with Tikhon Jelvis's "Lazy Dynamic Programming" article.

The basic idea is to **define an Array** of the computed values (like in DP), but **compute that array *recursively* from the Array itself** (rather than iteratively, as in imperative DP).

Memoising Fibonacci: lazy dynamic programming

Original

```haskell
fib :: Int -> Int
fib 0 = 0
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)
```

Memoised

```haskell
import Data.Array
fib' n = go n
  where go 0 = 0
        go 1 = 1
        go n = fibs ! (n - 1) + fibs ! (n - 2)
        fibs = listArray (0, n) [go x | x <- [0..n]]
```

Like ArrayList from Java - O(1) access. Index function is !

`go` = common name for a recursing helper subfunction (like `i` in a for loop)

Build from Array lazily from range of indices

Optimising Fibonacci

Our memoised Fibonacci is much faster, and can handle much higher values! But, we are not done…

```
import Data.Array
fib' max = go n
  where go 0 = 0
        go 1 = 1
        go n = fibs ! (n - 1) + fibs ! (n - 2)
        fibs = listArray (0, max) [go x | x <- [0..n]]
```

Our algorithm keeps an array of **all previous values** around, so it is now an O(n) space algorithm. But, you only need the previous *two* values of the sequence to compute the next one. **Fib can be O(1) space! Just because Haskell is lazy does NOT make it immune to regular algorithmic optimisation!**

Quiz

1. (1 points) What is a ‘base case’ in recursion?
   - **a.** First defined condition in the function
   - **b.** The first argument to the recursive function
   - **c.** The part of the function that calls itself (the function re-**base**s on itself)
   - **d.** The condition for the simplest input, which stops the recursion
2. (1 point) What is the difference between `foldl` and `foldr`
   - **a.** `foldl` is left associative
   - **b.** `foldl` uses the first element of the list as the initial `acc`, `foldr` uses the last
   - **c.** `foldl` is strict in the application of the operator
   - **d.** `foldl` is for lists only (`l` for `List`), and `foldr` is for `RList`s (also called `SnocList`s, which are like lists but right-associative)
3. (1 point) Which statement is most correct?
   - **a.** Recursion is totally different to loops and they can’t be compared
   - **b.** Recursion can implement for loops, but not while loops
   - **c.** Recursion can implement for loops and while loops, but not do-while loops
   - **d.** Recursion can implement any kind of loop

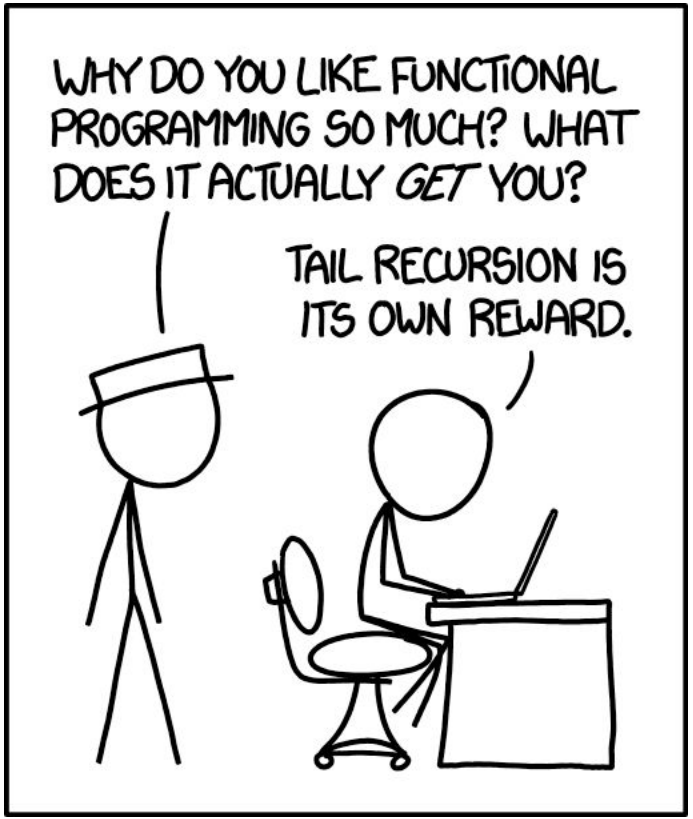


Tail recursion

Tail recursion is when a function always returns either (1) a simple value or (2) a call of itself (a **tail-call**). In many languages, this allows the function to use O(1) stack space.

In Haskell, tail recursion & tail-call elimination is **not *always* as important** due to the way that lazy evaluation is implemented with Thunks. See this page for more: "Tail recursion" on Haskell wiki.

However, it *can* help us here with fib!



"Functional programming combines the flexibility and power of abstract mathematics with the intuitive clarity of abstract mathematics." - Randall Munroe, https://xkcd.com/1270/

Optimising Fibonacci: tail-recursive

The + function is strict in both arguments, forcing evaluation and eliminating a potential large stack of thunks

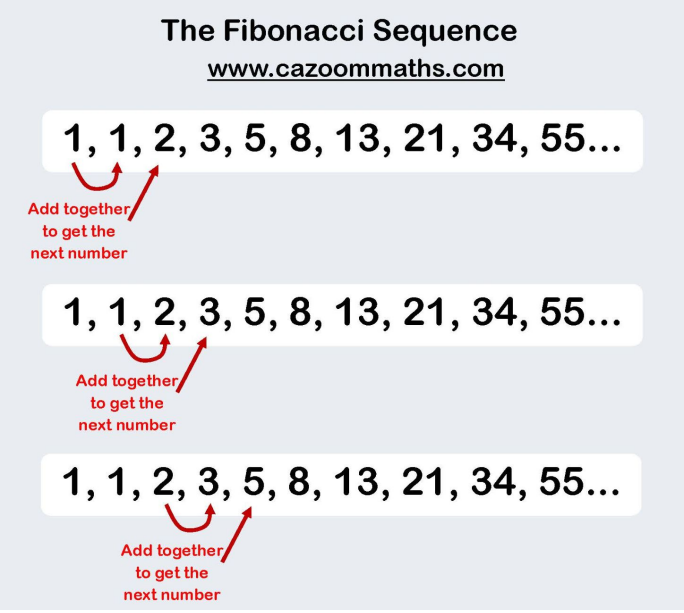
We can implement it tail-recursively as follows:

```
fib :: Int -> Integer
fib n = go 0 1 0
    where
  go n_2 n_1 cur_n
    | n == cur_n   = n_1
    | otherwise    = go n_1 (n_2 + n_1) (cur_n + 1)
```

Our function now takes O(1) stack space, as we only keep the last two results, passing them down the to the next `go` call.

Optimising Fibonacci: fold is the universal list combinator!

We can also think about fibonacci as a **sequence** once again and write it as a **fold** over the values of n. Here we call the value fib(n - 1) as `n_1`, and the value fib(n - 2) as `n_2`.




```
fib :: Int -> Integer
fib n = fst (foldl' go (0, 1) [1..n])
    where
  go (n_1, n_2) _ = (n_1 + n_2, n_1)
```

To recurse or to use a combinator?

You've just seen that can often implement things **either** with recursion **or** with some kind of combinator. You could implement fib recursively, or with foldl'. You could transform a list recursively (`transform (x:xs) = (x * 2) : (transform xs)`), or with map (`map (*2) xs`).

This is, ultimately, a matter of taste. Some programmers **prefer combinators** because each combinator **does one specific thing,** so it is quite obvious *what your code is trying to achieve* (for the map, example, it is MUCH clearer!)

Some even go as far as to say that **the more Haskell you know, the less recursion you write**, since you learn the **specific tools for each job.** But, never forget that **code is read more often than it is written, so keep code clear.**

Note: guarded recursion

Another important concept in Haskell when it comes to laziness and recursion is **guarded recursion.** You don't need to know the theoretical foundations, but essentially you want to **make sure that recursive calls happen in a data constructor, where possible.**

For instance, we could define the map function as

```
map f (x:xs) = (f x) : (map f xs)
```

This would ensure that we only need to evaluate the rest of the list (given by `map f xs`) **if and only if it is called for.** This is because, of course, Haskell is lazy.

Extra reading

If the differences between `foldl`, `foldr`, and `foldl'` confuse you, I recommend reading the following:

- https://blog.haskell.org/foldl-and-foldr/
- https://wiki.haskell.org/Foldr_Foldl_Foldl%27
- https://wiki.haskell.org/Performance/Strictness
- [AR https://wiki.haskell.org/index.php?title=Non-strict_semantics]
- [AR https://wiki.haskell.org/index.php?title=Lazy_vs._non-strict]

I am not calling this **assigned reading** as this is only necessary to read if you are confused specifically about this.

Exercises

(please remember that if you use AI to do the following, it no longer has any value. You are only cheating yourself.)

- Implement `foldr` recursively (call it `myfoldr`—`foldr'` is already taken)
- Implement merge sort recursively
- Read and understand "Fibonacci in Haskell" by Michael Jones



Assigned reading & assistance

As revision for today:

- “Fold, foldl’, foldr” on Haskell wiki
- (LYAH) Recursion (entire chapter)
- “Tail recursion in Haskell” on StackOverflow
- “Does Haskell have tail recursive optimisation?” on StackOverflow
- https://jelv.is/blog/Lazy-Dynamic-Programming/

If you have any queries, email them to me at mrqcae001@myuct.ac.za. I am happy to do office hours by appointment (also by email)